Exiting Methods with the Keyword return

The keyword `return` terminates the currently executed *Method* and returns to the calling *Method*.

Syntax

```
return
```

The keyword **return** terminates the currently executed *Method* and returns to the calling *Method*.

```
if ParallelStation.failed   // cannot process?
   return                   // return immediately
else
   ...
end
```

You can terminate a *Method* that returns a result with `return <expression>`. This has the same effect as `result := expression return`.

```
param n: integer -> integer // computes the factorial of n
if n = 0
   return 1
end
return n * self.execute(n-1)
```

See also

return [SimTalk]

result [SimTalk]

Video on YouTube

https://youtu.be/5t-wLNmKpbU?si=52AE7o19OdNm3Hg2&t=156

Unpublished work. © 2026 Siemens

Error Handling

Error handling, also known as **exception handling** in software development, enables you to react to runtime errors which occur within *Methods* while the source code is being executed.

Remarks

Error handling does, for example, make sense, if the simulation has to continue under all circumstances and to record the errors, which occur. It also makes sense if you want to take appropriate measures, if errors, which you expect to occur, do in fact occur.

To implement **error handling**, proceed as follows:

- For an object of type *Method*:

  Create a *user-defined attribute* of data type *method* or *object* and name it `ErrorHandler`. You have to create this attribute either in the *Method* itself or in the *Frame* into which you inserted the *Method*.

- For a *user-defined attribute* of data type *method*:

  Create another *user-defined attribute* in the object which has the *user-defined attribute(s)*. Assign the data type *method* or *object* this attribute and name it `ErrorHandler`.

The source code of the *ErrorHandler* could, for **example**, look like this:

```
param byref error: string,
      method_path: string,
      line_number: integer -> any
if error = "Division by zero."
   error := ""  // catch this error
   return 1e300 // return a value to the calling method
end
// route error message to the calling method
error := "error in " + method_path + ": " + error
```

When a runtime error occurs in a *Method*, *Plant Simulation* cancels executing that *Method* and calls the error handling method, the *ErrorHandler*, of the *Method*. If the *Method* does not have an *ErrorHandler*, *Plant Simulation* searches for an *ErrorHandler* within the call chain. If none of the *Methods* within the call chain have an *ErrorHandler*, *Plant Simulation* uses the standard error handling procedure, meaning that it opens the *Method Debugger*.

Parameters

Pass three parameters to the *ErrorHandler*:

- The error message.
- The path of the faulty *Method*.
- The number of the line of code in which the error occurred.

Unpublished work. © 2026 Siemens

If you assign an empty string "" to the error message variable, *Plant Simulation* assumes that you successfully took care of the error. *Plant Simulation* will not continue executing the faulty *Method*. Instead, *Plant Simulation* jumps back to that *Method*, that called the *Method*, which has the *ErrorHandler*.

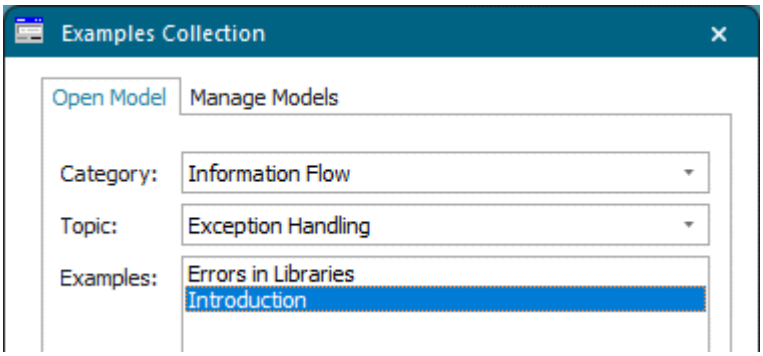
If the *Method*, which has the *ErrorHandler*, returns a value, the *ErrorHandler* should also do so.

If you cannot react appropriately to the error because, for example, an error occurred, which you did not expect, you should not delete the error message. You can change it at will though. If you do not delete the error message, *Plant Simulation* assumes that you did not take care of the error, and passes the error message to the calling *Method*. If the call chain contains another *Method* with an *ErrorHandler*, *Plant Simulation* calls this *ErrorHandler* with the changed error message. Otherwise, *Plant Simulation* uses the standard error handling procedure with the changed error message, meaning that it opens the *Method Debugger*.

Note

If another runtime error occurs during error handling, *Plant Simulation* does not re-execute the error handling *Method*.

Compare the example model in the **Examples Collection**.



See also

setErrorHandler [SimTalk]

getCallStack [SimTalk]

throwRuntimeError [SimTalk]

Calculate Values with a Formula

Unpublished work. © 2026 Siemens